



1. Problem Title & Link

Title: LeetCode 48 : Rotate Image

Link: [LeetCode 48 - Rotate Image](#)

2. Problem Statement (Short Summary)

You are given an $n \times n$ matrix representing an image.

Rotate the image by:

90° clockwise

Rules:

- Rotate **in-place**
- Do not create another matrix

In simple terms:

Convert:

1 2 3

4 5 6

7 8 9

Into:

7 4 1

8 5 2

9 6 3

without using extra matrix space.

3. Examples (Input → Output)

Example 1

Input:

matrix=

```
[  
  [1,2,3],  
  [4,5,6],  
  [7,8,9]  
]
```

Output:

```
[  
  [7,4,1],  
  [8,5,2],  
  [9,6,3]  
]
```

**Example 2**

Input:

matrix=

```
[  
  [5,1,9,11],  
  [2,4,8,10],  
  [13,3,6,7],  
  [15,14,12,16]  
]
```

Output:

```
[  
  [15,13,2,5],  
  [14,3,4,1],  
  [12,6,8,9],  
  [16,7,10,11]  
]
```

4. Constraints

$n == \text{matrix.length}$

$n == \text{matrix}[i].\text{length}$

$1 \leq n \leq 20$

$-1000 \leq \text{matrix}[i][j] \leq 1000$

Special restrictions:

- Must rotate in-place
- Avoid extra matrix

5. Core Concept (Pattern / Topic)

Matrix Manipulation + Transpose

6. Thought Process (Step-by-Step Explanation)

Brute Force

Create another matrix:

Rule:

$\text{new}[j][n-1-i]$



=

old[i][j]

Time:

$O(n^2)$

Space:

$O(n^2)$

Extra matrix violates the condition.

Optimized Thinking

Observation:

90° clockwise rotation can be achieved in two steps:

Step 1:

Transpose matrix

Rows become columns

Step 2:

Reverse every row

Why?

Original:

1 2 3

4 5 6

7 8 9

Transpose:

1 4 7

2 5 8

3 6 9

Reverse rows:

7 4 1

8 5 2

9 6 3

7. Visual / Intuition Diagram (ASCII Diagram)

Original:

1 2 3

4 5 6

7 8 9

Transpose:

1 4 7



2 5 8

3 6 9

Reverse rows:

7 4 1

8 5 2

9 6 3

Final rotation:

90° clockwise

8. Pseudocode (Language Independent)

n=matrix.length

for i=0 to n-1:

 for j=i+1 to n-1:

 swap(
 matrix[i][j],
 matrix[j][i]
)

for each row:

 reverse(row)

9. Code Implementation

Python

class Solution:

 def rotate(
 self,
 matrix):

 n=len(matrix)

 for i in range(n):



```
for j in range(  
    i+1,  
    n  
):
```

```
    matrix[i][j],\  
    matrix[j][i]=\  
    matrix[j][i],\  
    matrix[i][j]
```

```
for row in matrix:
```

```
    row.reverse()
```

Java

```
class Solution {  
  
    public void rotate(  
        int[][] matrix) {  
  
        int n=  
            matrix.length;  
  
        for(int i=0;  
            i<n;  
            i++){  
  
            for(int j=i+1;  
                j<n;  
                j++){  
  
                int temp=  
                    matrix[i][j];  
  
                matrix[i][j]=
```



```
        matrix[j][i];

        matrix[j][i]=
            temp;
    }
}

for(int i=0;
    i<n;
    i++){

    int left=0;
    int right=n-1;

    while(left<right){

        int temp=
            matrix[i][left];

        matrix[i][left]=
            matrix[i][right];

        matrix[i][right]=
            temp;

        left++;
        right--;
    }
}
}
```

10. Time & Space Complexity

Time Complexity:

$O(n^2)$

Reason:



Every element is visited once.

Space Complexity:

$O(1)$

Reason:

Only swapping operations used.

11. Common Mistakes / Edge Cases

1. Creating extra matrix

Violates:

in-place requirement

2. Incorrect transpose loop

Wrong:

for $j=0$

Correct:

for $j=i+1$

Avoids duplicate swaps.

3. Reversing columns instead of rows

4. Missing:

1x1 matrix

Input:

[[5]]

Output:

[[5]]

12. Detailed Dry Run (Step-by-Step Table)

Input:

```
[
[1,2,3],
[4,5,6],
[7,8,9]
]
```

Step 1: Transpose

Swap	Matrix
p	



(2,4)	[1,4,3]
(3,7)	[1,4,7]
(6,8)	Complete d

Result:

1 4 7

2 5 8

3 6 9

Step 2: Reverse Rows

Row	Result
1 4 7	7 4 1
2 5 8	8 5 2
3 6 9	9 6 3

Final:

[
[7,4,1],
[8,5,2],
[9,6,3]
]

13. Common Use Cases (Real-Life / Interview)

Image rotation systems

Computer graphics

Matrix transformations

Game board rotations

14. Common Traps (Important!)

1. Swapping twice during transpose
2. Reversing columns instead of rows
3. Using extra matrix
4. Wrong loop boundaries

15. Builds To (Related LeetCode Problems)

Progression:



- LC 48 → Rotate Image
- LC 54 → Spiral Matrix
- LC 73 → Set Matrix Zeroes
- LC 59 → Spiral Matrix II
- LC 240 → Search a 2D Matrix II

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Extra Matrix	$O(n^2)$	$O(n^2)$	Simple
Layer-by-layer Rotation	$O(n^2)$	$O(1)$	More complex
Transpose + Reverse	$O(n^2)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

Transposing converts rows into columns, and reversing rows aligns the elements into their rotated positions. Together, these operations exactly simulate a 90° clockwise rotation in-place.

18. Variations / Follow-Up Questions

1. Rotate 90° anti-clockwise
2. Rotate 180°
3. Rotate rectangular matrix
4. Rotate image multiple times
5. Rotate matrix layer by layer
- 6.